

TRABALHO PRÁTICO 1 - COMPUTAÇÃO GRÁFICA

Aluna: Yasmin Viegas - 800989

Professora: Rosilane Mota

Link para vídeo no google drive:  TP1_Yasmin.mp4

Como executar o trabalho

Executar .exe em: TP1_Yasmin\TP_CG\dist\ComputacaoGrafica\ComputacaoGrafica.exe

Ou se preferir rodar o Python:

- Requisitos: Python 3.8 ou superior com pip disponível
- Entrar na pasta TP_CG (onde está o código) e dentro dela:
 - Instalar dependências: **pip install -r requirements.txt**
 - Executar: **python main.py**

Descrição do trabalho

Este trabalho implementa uma aplicação gráfica interativa de desenho 2D, desenvolvida em Python com PyQt5. A aplicação permite desenhar primitivas geométricas, aplicar transformações e recortar o conteúdo por uma janela retangular, sem uso de bibliotecas de desenho de alto nível, já que todos os pixels são calculados manualmente pelos algoritmos implementados.

main.py

Define a janela principal (JanelaPrincipal), monta a toolbar com os botões de modo (Reta, Círculo, Polígono, Selecionar, Recortar), os seletores de algoritmo de reta e de recorte, e o painel lateral com os controles de transformação geométrica (translação, rotação, escala e reflexões). Quando o usuário clica em algum desses botões, a main captura os valores dos spinboxes e chama os métodos correspondentes em paint.py, passando o centro da tela como pivot das transformações.

paint.py

É a área onde o desenho acontece. Herda de QWidget e usa um QPixmap como buffer em memória: os pixels são escritos nesse buffer e depois copiados na tela a cada paintEvent. Gerencia os eventos de mouse (clique, arraste, soltura) para capturar pontos de retas, círculos, polígonos, retângulos de seleção e de recorte. No redesenhar_tudo, percorre todas as primitivas do gerenciador e as rasteriza novamente do zero, aplicando destaque laranja nos itens selecionados. Também contém a lógica de transformação de círculos (que são armazenados como dicionário {xc, yc, r, cor} e não como pontos, então precisam de tratamento separado).

algoritmos/formas.py

Define as estruturas de dados da aplicação: Ponto, Reta e Polígono (como dataclasses), e a classe Desenho (exportada como GerenciadorDesenho), que é o ponto central de todas as primitivas. O gerenciador mantém listas originais de retas, círculos e

polígonos que nunca são modificadas pelo recorte. As propriedades `retas`, `círculos` e `arestas_poligonos_visiveis` aplicam o filtro de recorte na hora de retornar os dados para renderização. **O recorte é não-destrutivo por design**. Também implementa a seleção por retângulo, usando Cohen-Sutherland internamente para testar se cada primitiva toca a região selecionada.

algoritmos/rasterizacao.py

Implementa os três algoritmos de rasterização exigidos. O **DDA** calcula incrementos fracionários em x e y a cada passo e arredonda para o pixel mais próximo utilizando o ponto flutuante. O **Bresenham** para retas usa apenas aritmética inteira: mantém um parâmetro de decisão p que determina se o pixel seguinte sobe ou não no eixo secundário, com dois casos ($dx > dy$ e $dy \geq dx$) para tratar todas as inclinações. O **Bresenham para círculos** explora a simetria de 8 octantes e a cada iteração calcula apenas um ponto e replica para os outros sete, usando um parâmetro de decisão $p = 3 - 2r$ inicializado antes do loop. Todos os três recebem um callback `desenhar_pixel` na construção, assim esse arquivo apenas calcula coordenadas.

algoritmos/recorte.py

Implementa os dois algoritmos de recorte de segmentos de reta. O **Cohen-Sutherland** atribui a cada ponto um código de 4 bits representando em qual região fora da janela ele está (esquerda, direita, abaixo, acima). Se ambos os pontos têm código zero, o segmento está completamente dentro. Se o AND dos dois códigos é diferente de zero, está completamente fora. Caso contrário, recorta iterativamente o ponto externo contra a borda indicada pelo bit ativo até convergir. O **Liang-Barsky** trata a reta de forma paramétrica ($P1 + u \cdot (P2 - P1)$), com $u \in [0, 1]$ e para cada uma das quatro bordas calcula o valor de u onde a reta a atravessa, atualizando os limites $u1$ (entrada) e $u2$ (saída). Se em algum momento $u1 > u2$, o segmento está fora. Ao final, recalcula as coordenadas dos extremos visíveis com base nos valores finais de $u1$ e $u2$. Ambos retornam um novo objeto `Reta` com os pontos recortados, ou `None` se o segmento estiver completamente fora.

algoritmos/transform.py

Implementa as seis transformações geométricas como métodos estáticos da classe `Transformador`. Todas funcionam pelo mesmo padrão: definem uma função local que transforma um `Ponto`, e passam essa função para `_aplicar_a_pontos`, que distribui a transformação para os dois extremos de uma `Reta` ou para todos os vértices de um `Polígono`.

- **Translação**: Soma dx e dy diretamente às coordenadas.
- **Rotação**: Translada o ponto para a origem (subtrai o pivot), aplica a matriz de rotação com seno e cosseno, translada de volta.
- **Escala**: Mesmo padrão da rotação, translada para a origem, multiplica por sx/sy , volta.
- **Reflexão X** (espelho horizontal): Inverte y em relação ao eixo $y = \text{eixo_y}$, usando $2 \cdot \text{eixo_y} - y$.
- **Reflexão Y** (espelho vertical): Inverte x em relação ao eixo $x = \text{eixo_x}$.
- **Reflexão XY** (reflexão pontual): Inverte as duas coordenadas em relação ao ponto central, equivalente a uma rotação de 180° .